

LLM 预训练之 Loss / Training Objective

Cross-Entropy / Loss Mask / Reduction / Auxiliary Objectives / Multi-Token Prediction • 深度研究手册

从“预测下一个 Token”到真正可训练的监督系统：目标函数如何塑造梯度、训练稳定性与模型能力

核心视角：Training Objective = supervision definition + token selection + reduction semantics + auxiliary constraints + numerical implementation

Research cut-off: 2026-08-13 | 01-B-Agent



Training objective is defined not only by CE, but also by which tokens are supervised and how they are reduced.

图 1 从 Sequence 到 Gradient：Cross-Entropy 只是中间一环；loss mask 与 reduction 决定哪些 token 以多大权重进入梯度。

执行摘要：Loss 真正决定了什么

现代 decoder-only LLM 的核心目标非常简单：在每个位置根据左侧上下文预测下一个 token，并最小化 token-level negative log-likelihood / cross-entropy。PaLM 对这一类模型描述就是标准 left-to-right objective：给定 preceding tokens 预测 next token。[1] 但工程上“loss”远不止一个公式。Sequence packing 后哪些边界 token 要忽略、padding 是否计入、不同长度样本怎样归一化、多个 data-parallel rank 如何做全局 token normalization、是否加入 z-loss / MoE balancing / MTP 等辅助项，都会改变实际梯度。

层次	核心问题	错误实现的后果
Supervision	label 是哪一个未来 token？是否做 shift？	off-by-one、训练目标错位
Mask	哪些位置进入 loss？EOD/PAD/跨文档边界如何处理？	无意义 token 或边界污染梯度
Reduction	按 token、sequence、microbatch 还是 global batch 平均？	长度/设备/packing 改变隐式样本权重
Auxiliary objectives	z-loss / MoE balance / MTP 等加多大权重？	主任务被辅助项压制，或稳定性不足
Numerics	log-softmax、vocab parallel、低精度如何实现？	OOM、NaN/Inf、通信瓶颈、loss spike

1. Core Objective：Next-Token Prediction 到底优化什么

对于 token 序列 $x_1, \dots, x_T$ ，decoder-only LLM 建模联合概率：

$$P(x_1, \dots, x_T) = \prod_t P(x_t \mid x_{<t})$$

训练时通常等价于最小化所有有效 token 的负对数似然：

$$L_{NTP} = - (1 / N_{\text{valid}}) \sum_t m_t \cdot \log P_\theta(x_t \mid x_{<t})$$

其中 $m_t \in \{0,1\}$ 是 loss mask, $N_{\text{valid}} = \sum_t m_t$ 。这个写法比只写 “CrossEntropy(logits, labels)” 更接近真实训练系统，因为它把监督位置与归一化规则显式暴露出来。

Know-why: NTP 的监督信号密度极高。长度 L 的一条 sequence 在一次 forward 中可同时产生约 L 个 token prediction targets; causal mask 让模型无法偷看未来，但 GPU 仍能并行计算整段训练序列。

2. Label Shift：最简单也最容易出错的一步

位置	模型输入	训练标签	语义
t=0	x_0	x_1	看到第 0 个 token，预测第 1 个
t=1	x_1	x_2	隐藏状态 h_1 用来预测后继
...	...	...	所有位置一次 forward 并行训练
末尾	$x_{\{L-1\}}$	x_L / EOS	取决于 sequence builder 是否提供 L+1 token

工程上常见两种接口：数据集直接返回 tokens 与 labels；或模型内部对同一条 token 序列做 shift。无论哪一种，都必须和 Sequence Construction 中的 L+1、EOS/EOD、padding、document boundary 设计完全一致。Megatron GPT 数据路径显式传递 labels 与 loss_mask，并提供 eod_mask_loss、inter-document masking 等配置来控制边界语义。[6]

3. Cross-Entropy：为什么它是自然语言建模的主损失

$$CE_t = -\log \text{softmax}(z_t)[y_t] = -z_{\{t,y\}} + \log \sum_j \exp(z_{\{t,j\}})$$

它等价于对真实 one-hot token 分布与模型预测分布做 cross-entropy；PyTorch 文档也明确指出，类别索引形式的 CrossEntropyLoss 等价于 LogSoftmax + NLLLoss。[5]

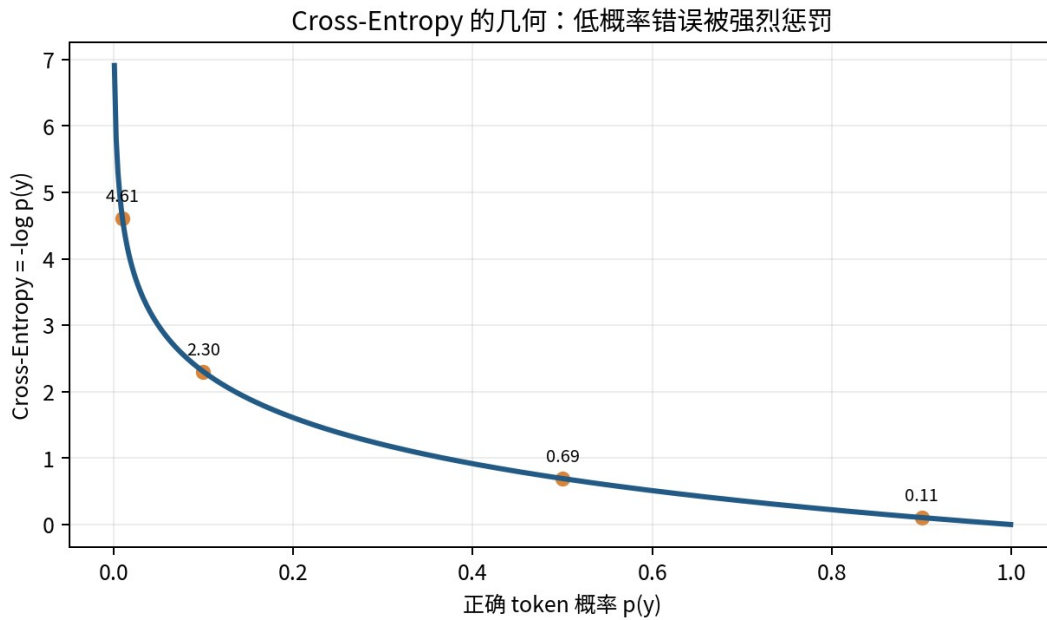


图 2 Cross-Entropy 对低正确概率进行强惩罚； $p(y)$ 从 0.1 提升到 0.9 的收益不是线性的。

梯度层面更重要：若 $p = \text{softmax}(z)$ ，one-hot 标签为 y ，则对 logits 的梯度是 $p - y$ 。也就是说，每个位置不仅推动正确 token logit 上升，还同时压低整个 vocabulary 的其他候选；这也是为什么 LM Head 与 Cross-Entropy 会把 vocabulary-level supervision 传回 backbone。

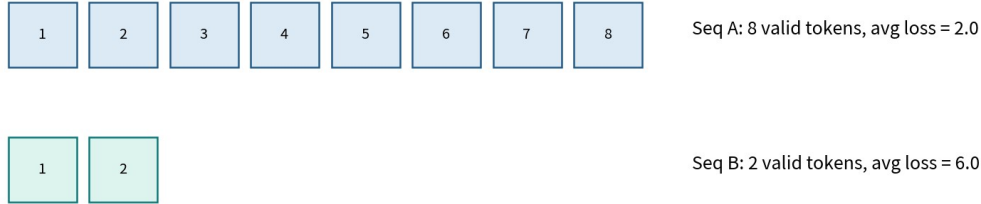
4. Loss Mask：哪些 token “不应该教模型”

场景	典型 mask 决策	Know-why
Padding	PAD 不计入 loss	否则 batch padding 比例会改变优化目标
EOD/EOS	可训练，也可配置忽略	是否让模型显式学习文档终止是 recipe 选择
Packed docs	通常保留各文档内部 token；边界 token 取决于 packing 语义	物理拼接不代表逻辑上必须跨文档监督
SFT-style mixed data	prompt / system / tool tokens 可选择 mask	只让目标 response 贡献 loss
损坏/无效 span	动态设 $m_t=0$	避免已知坏标注进入梯度

对纯预训练而言，最常见的是 padding 与 EOD 语义；但当数据开始混入 FIM、代码结构、合成样本或 mid-training 格式时，loss mask 会逐渐成为 objective compiler 的一部分。

5. Reduction Semantics：一个“平均”就能改变训练分布

为什么 reduction 语义重要：variable-length / padding 场景



Sequence-mean: $(2 + 6)/2 = 4.0$

Token-mean: $(8 \times 2 + 2 \times 6)/10 = 2.8$

同一批数据，仅 reduction 规则不同，梯度权重就不同：短序列在 sequence-mean 下被过度放大。

图 3 Sequence-mean 与 Token-mean 会给短序列不同的隐式权重；variable-length batch 中必须显式选择。

在 fixed-length、无 padding 时，sequence mean、token mean 往往看起来差别不大；一旦使用 packing、variable-length、SFT-like masks 或 distributed token imbalance，它们就可能明显不同。Megatron 当前 loss_func 会先对有效 token loss 求和，并同时返回 num_tokens；其 TransformerConfig 还提供 calculate_per_token_loss，用实际 non-padding token 数而不是假设所有 token 都有效。[6][7]

Reduction	形式	隐式权重
Token mean	$\Sigma \text{ token loss} / \Sigma \text{ valid tokens}$	每个有效 token 等权
Sequence mean	$\text{mean}_i(\text{mean}_t \text{ loss}_{it})$	每条 sequence 等权，短序列 token 权重大
Microbatch mean	$\text{mean}_b(\text{loss_microbatch})$	受 microbatch token 数影响
Global token mean	$\text{all-reduce } \Sigma \text{ loss} / \text{all-reduce } \Sigma \text{ tokens}$	distributed variable-length 场景最清晰

6. Perplexity：Loss 的可解释外壳，但不要跨 Tokenizer 乱比

$$\text{PPL} = \exp(\text{average negative log-likelihood})$$

如果 loss 使用自然对数并按 token 平均，PPL 可以直观理解为模型在每个位置面对的“有效候选不确定性”的指数形式。它对同 tokenizer / 同数据分布的训练曲线非常有用。

但跨 tokenizer 比较 token-level PPL 会产生结构性偏差：不同 tokenizer 把同一段原始文本切成不同数量、不同粒度的事件。更可靠的跨 tokenizer 比较应固定 raw text，并考虑 bits-per-byte / bits-per-character 等归一化，或直接比较 downstream capability。

7. Loss 与 Scaling Laws：为什么一个 scalar 能指导十亿美元级训练

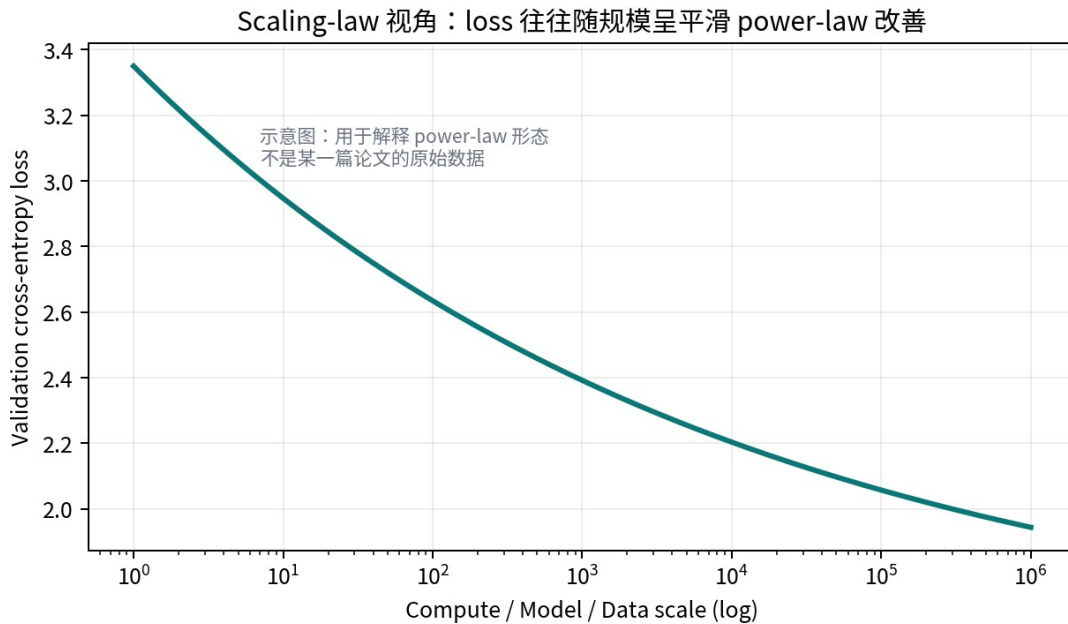


图 4 示意：Kaplan 等研究观察到 cross-entropy loss 随模型规模、数据量与 compute 呈平滑 power-law 关系；图中曲线仅用于机制说明。

Kaplan et al. 2020 系统研究了语言模型 cross-entropy loss 的 scaling，发现 loss 随 model size、dataset size、compute 在很大范围内呈平滑 power-law。[2] Hoffmann et al. 2022 (Chinchilla) 进一步用 400+ 模型重新估计 compute-optimal 的参数量与训练 token 配比，并显示在固定 compute 下，模型规模与训练 token 都需要一起增长。[3]

Know-why: validation loss 是一个低方差、连续、训练过程可频繁观测的 proxy，因此非常适合做 scaling law 和 early prediction；但它不是 downstream capability 的完美替代。不同数据 mixture、tokenizer、objective 或 benchmark 可能让“同 loss”对应不同能力结构。

8. Stability Regularization：Cross-Entropy 之外为什么还要加 z-loss

$$L_z = c_z \cdot \text{mean}(\text{logsumexp}(z)^2)$$

T5X 的稳定 cross-entropy 实现把 z-loss 定义为对 softmax 归一化常数 $\log Z$ 的平方惩罚，并给出两个目的：抑制 logits 远离 0 以减少 bfloat16 roundoff 问题，以及鼓励 logits 接近 normalized log-probabilities。[8] ST-MoE 将同一思想用于 router logits，作为 router z-loss，强调其训练稳定性收益。[10]

方法	作用对象	主要目的	注意点
LM-head z-loss	最终 vocabulary logits	控制 logit drift / 数值稳定	会引入正则强度超参
Router z-loss	MoE router logits	稳定 expert routing softmax	不是语言建模主目标
Logit soft-capping	最终 logits	硬限制极端值	会改变 logits 几何与梯度
Gradient clipping	参数梯度	防止极端 update	只能处理后果，不等价于 logits regularization

2026 年 Output Embedding Centering 进一步把 output logit divergence 解释为输出 embedding geometry 问题，并提出 μ -centering / μ -loss；这是值得关注的前沿方向，但目前证据成熟度低于 CE/z-loss 的长期实践。[16]

9. Auxiliary Losses：总目标常常不是只有 LM Loss

$$L_{\text{total}} = L_{\text{LM}} + \lambda_{\text{balance}} L_{\text{balance}} + \lambda_z L_z + \lambda_{\text{MTP}} L_{\text{MTP}} + \dots$$

Switch Transformer 使用 differentiable auxiliary load-balancing loss，让 expert token fraction 与 router probability 更接近均匀分布；论文中该辅助项与主 cross-entropy objective 共同训练。[9] ST-MoE 又在此基础上加入 router z-loss。[10]

关键工程原则：辅助 loss 解决的通常是“系统约束”或“训练稳定性”，不是在替代语言建模目标。权重过大时，模型会为了路由均衡或 logit 正则牺牲 next-token likelihood；权重过小时则起不到约束作用。因此需要同时记录 raw LM loss 与各 auxiliary loss，而不是只报告一个 total loss。

10. Beyond Next Token：FIM、MTP 与 Objective Mixtures



图 5 预训练目标需要区分：主语言建模目标、数据变换以及系统性辅助约束。

Objective	做法	保留 NTP 吗？	核心价值
Standard NTP	预测下一个 token	是，主目标	通用 decoder-only foundation
FIM	把 middle span 移到末尾/重排，再做 autoregressive CE	本质仍是 AR CE	获得 infilling 能力；论文显示高比例 FIM 不显著伤害 LTR 能力 [12]
Multi-Token Prediction	同一 hidden state 预测多个未来 token	通常 NTP + 多未来 head	Meta 2024 报告更高 sample efficiency, 13B 在 HumanEval/MBPP 有明显提升 [11]
DeepSeek-V3 MTP	在预训练中采用 multi-token prediction objective	是扩展目标	技术报告把 MTP 作为性能增强组件 [13]
Span corruption / UL2	遮盖 span 或混合 denoiser objective	不是标准 causal NTP	更偏 encoder-decoder / unified objective 体系 [14][15]

一个重要区分：FIM 经常只是“数据变换”，loss 仍是标准 autoregressive CE；MTP 则真的增加了未来预测 heads / objectives；UL2 则进一步把多种 denoising objective 混合。不要把三者都笼统称为“换 loss”。

11. Production Cross-Entropy：数学一样，系统实现可以完全不同

当 vocabulary 达到 128K/256K、sequence 很长、tensor parallel 很大时，直接显式物化完整 [tokens, vocab] logits 再做 softmax/CE 可能成为显存与通信热点。Megatron Core 提供 fused_vocab_parallel_cross_entropy：在 vocabulary dimension 被 tensor-parallel 切分时，通过 global max、目标 logit、sum-exp 与梯度计算保持 exact cross-entropy。[4]

系统问题	典型工程方案	不改变什么
Large vocab logits memory	fused CE / chunked CE / cut CE	理论 CE objective
Tensor Parallel	vocab-parallel logits + collective reductions	全局 softmax 概率
Low precision	FP32 logsumexp / stable kernel	最终 loss 语义
Variable-length	sum loss + global valid-token count	每个有效 token 的权重定义
Pipeline parallel	只在 last stage 计算 LM loss	backbone 的全局梯度链

Know-how：优化 CE kernel 时必须把“exact objective”与“approximate softmax”分开。前者只是改执行计划；后者（sampled/hierarchical/adaptive softmax）会改变训练问题本身。

12. Loss Observability：只看一条 train loss 曲线远远不够

监控维度	指标	能发现什么
Primary	train/val NLL、PPL	总体学习进度
Token buckets	按频率/语言/域/长度分桶 loss	长尾、语言或领域被系统性忽略
Mask	valid-token ratio、PAD/EOD supervised ratio	packing / data pipeline 错误
Logits	RMS、max、logsumexp、entropy	output drift、过度自信、z-loss 风险
Stability	grad norm、update norm、NaN/Inf、spiky loss	训练崩溃前兆
Auxiliary	balance loss、router z-loss、MTP loss	辅助目标是否压过主 LM loss
Distributed	per-rank token count / loss sum	sharding 与 variable-length 不平衡

Megatron 当前实现甚至定义了 spiky loss 检查：把“异常大 loss”作为 rerun/recovery 的检测信号。[6] 但 loss spike 并不一定是 objective 本身的问题；Spike No More 等研究把其根源更多归结到 Jacobian / gradient dynamics 与残差尺度。[17]

13. 严格 Ablation：如何判断“新的 Loss 更好”

- 固定 tokenizer、raw data、data order、total tokens、optimizer、LR schedule、batch token count；否则 loss 改动与训练预算耦合。
- 若引入 auxiliary objective，必须分别报告 L_{LM} 、 L_{aux} 、 L_{total} ；不要用 total loss 直接和纯 NTP loss 横比。
- 同时比较 validation NLL 与 downstream capability：新 objective 可能牺牲 next-token likelihood，却提升 code infilling / reasoning 等目标能力。
- 固定参数/FLOPs：MTP 增加 output heads 时，应区分“更多参数”与“objective 更好”。
- 在短训 proxy 与长训 horizon 都验证：有些正则/辅助项只在长训练时体现稳定性收益。
- 记录 token-level reduction、mask policy、distributed normalization；这些实现细节必须进入实验配置，而不是藏在 dataloader/loss_func 里。

实验问题	建议对照	核心读数
CE vs CE+z-loss	同 data/order/LR	loss spike、logit RMS、val NLL
NTP vs FIM mix	固定 raw token budget	LTR PPL、FIM benchmark、code completion
NTP vs MTP	参数/FLOPs 匹配	sample efficiency、coding/reasoning、inference acceleration
token-mean vs sequence-mean	同 variable-length batch	长/短样本梯度权重、val loss
MoE aux coeff sweep	固定 expert/router config	load balance、drop rate、LM loss、throughput

14. Production Blueprint：把 Loss 当作一层“Objective Runtime”

推荐把训练目标拆成显式的可配置流水线，而不是在 `model.forward()` 中硬编码一个 scalar：

labels → token_loss → masks → named auxiliary losses → weighted sum → global token normalization → backward

模块	建议接口	必须记录
Label compiler	shift / FIM / MTP targets	target type、offset、special-token policy
Mask compiler	PAD/EOD/document/prompt masks	valid tokens、各 mask 原因计数
Primary loss	exact CE / NLL	sum loss + num valid tokens
Aux registry	name → raw value → coefficient	每项 raw/weighted loss
Reducer	token/global normalization	DP/CP/PP group 与 token count
Numerics	dtype / fused kernel / vocab parallel	logit max、NaN/Inf、kernel version
Evaluator	val NLL + capability suite	同 tokenizer / raw-text normalization

15. Know-Why / Know-How：最终沉淀

Know-Why	Know-How
NTP 为什么强？	每个 token 都产生监督，目标与生成过程完全一致。
为什么 mask 是 objective 的一部分？	m_t=0 的位置没有梯度；“不训练什么”与“训练什么”同样重要。
为什么 reduction 不能随便 mean？	它定义样本/序列/token 的隐式权重。
为什么 validation loss 重要？	连续、低方差、可频繁测量，是 scaling law 的核心 proxy。
为什么 loss 不能代表全部能力？	同一 NLL 可对应不同数据、tokenizer、objective 与能力结构。
为什么需要 auxiliary losses？	解决稳定性、稀疏路由、未来预测等 NTP 本身不约束的问题。
为什么数值实现也属于 loss engineering？	large vocab、低精度和并行会让 exact CE 成为系统瓶颈。
最重要的实验原则？	固定 token budget 与归一化语义，分别记录 raw LM loss 与 auxiliary loss。

核心参考资料与证据等级

- [1] Chowdhery et al., PaLM: Scaling Language Modeling with Pathways, arXiv:2204.02311. 证据：大型 decoder-only LLM 标准 left-to-right next-token objective。
- [2] Kaplan et al., Scaling Laws for Neural Language Models, arXiv:2001.08361. 证据：cross-entropy loss 的经验 scaling law。
- [3] Hoffmann et al., Training Compute-Optimal Large Language Models, arXiv:2203.15556. 证据：Chinchilla compute-optimal 参数/token 配比。
- [4] NVIDIA Megatron Core, fused_vocab_parallel_cross_entropy / tensor_parallel.cross_entropy docs (2026). 证据：large-vocab exact CE 的 tensor-parallel/fused 实现。
- [5] PyTorch CrossEntropyLoss documentation (2026). 证据：CE、ignore_index、reduction、label_smoothing 的标准定义。
- [6] NVIDIA Megatron-LM pretrain_gpt.py (2026). 证据：loss_mask、sum loss、num_tokens、spiky-loss 检测等 production 语义。
- [7] NVIDIA Megatron Core TransformerConfig.calculate_per_token_loss (2026). 证据：按真实 non-padding token 计算 CE。
- [8] T5X losses.cross_entropy_with_logits documentation/source. 证据：final-logit z-loss 的定义与数值动机。
- [9] Fedus et al., Switch Transformers, JMLR 2022 / arXiv:2101.03961. 证据：MoE auxiliary load-balancing loss。
- [10] Zoph et al., ST-MoE, arXiv:2202.08906. 证据：router z-loss 与 sparse-model training stability。
- [11] Gloeckle et al., Better & Faster Large Language Models via Multi-token Prediction, arXiv:2404.19737. 证据：MTP sample efficiency 与 coding gains。
- [12] Bavarian et al., Efficient Training of Language Models to Fill in the Middle, arXiv:2207.14255. 证据：FIM data transform 与 LTR 能力兼容。
- [13] DeepSeek-AI, DeepSeek-V3 Technical Report, arXiv:2412.19437. 证据：MTP objective 与 auxiliary-loss-free balancing。
- [14] Raffel et al., T5, JMLR 2020. 证据：span corruption / text-to-text pretraining objective。
- [15] Tay et al., UL2: Unifying Language Learning Paradigms, arXiv:2205.05131. 证据：Mixture-of-Denoisers objective。
- [16] Stollenwerk et al., Output Embedding Centering for Stable LLM Pretraining, arXiv:2601.02031. 前沿预印本：output-logit divergence 与 μ -centering/ μ -loss。
- [17] Takase et al., Spike No More: Stabilizing the Pre-training of Large Language Models, arXiv:2312.16903. 证据：loss spikes 与 gradient/Jacobian dynamics。
- 说明：2026 年前沿预印本仅用于展示正在形成的新解释，不与已被长期复现的 Cross-Entropy / NTP / Scaling-Law 证据等量齐观。